

Exact Finite-State Coverage of Hellman and Distinguished-Point Time–Memory Tradeoffs: A Reproducible Study on Reduced-DES Mappings

Ali Mkhida
Algorizk Labs

28 August 2026

Abstract

Cryptanalytic time–memory tradeoffs replace repeated exhaustive search by precomputed chains of an iteration function. Their success depends not on the nominal number of generated chain positions, but on the number of distinct states actually represented after self-intersections and chain merging.

We present a reproducible finite-state study of Hellman and bounded distinguished-point tables for a concrete family of reduced-DES iteration maps. The artifact computes exact state-space coverage and exact key recovery, and provides independent Java and Rust implementations with shared semantic test vectors. In the current $N = 2^{16}$ configuration with $m = 1024$ Hellman chains of length $t = 64$, the measured median exact coverage is approximately 16.8%. Applying the established random-function coverage recurrence of Ma and Hong predicts approximately 16.651% coverage for the same parameters, showing that the observed loss is consistent with classical random-mapping theory rather than with the independent-sampling heuristic $1 - e^{-mt/N}$.

The final version of this report extends this comparison across parameter sweeps, random-function baselines, distinguished-point stopping parameters, and a shared Java/Rust experiment plan. All numerical claims are tied to committed inputs, outputs, and environment metadata.

1 Introduction

A time–memory tradeoff (TMTO) moves work from the online phase of a search problem into a reusable preprocessing phase. In Hellman’s classical construction [Hel80], a large key or state space is represented by iteration chains while only chain endpoints are retained. The construction therefore occupies a middle ground between exhaustive search, which uses little memory and large online time, and complete table lookup, which uses large memory and very small online time.

The central implementation issue is that a chain table does not contain mt independent samples. Once two iterates of the same mapping meet, their future trajectories coincide; a chain can also revisit one of its own previous states. Consequently, the effective coverage of a table is the number of *distinct* represented states, not its nominal number of positions. This phenomenon is part of the classical TMTO literature rather than a new observation: Hellman’s analysis, later work on random mappings, success probability, perfect tables, false alarms, and rainbow reductions all treat different aspects of collisions, merging, and coverage [Hel80, FO90, Oec03, AJO08, MH09, Hon08].

The purpose of this paper is therefore narrower and experimentally explicit. We ask how accurately established random-function TMTO theory describes a *concrete finite cryptographic mapping* when every represented state can be enumerated exactly, and whether independent implementations reproduce the same semantics and finite-state statistics.

Concrete iteration map. For a reduced state x , a canonical DES-key encoding $K(x)$, a fixed plaintext P , encryption E , and reduction R , we study

$$F(x) = R(E_{K(x)}(P)).$$

DES is obsolete and is used only as a compact deterministic fixture. The state dimension is deliberately reduced so that exact coverage, cycles, endpoint multiplicities, and recovery behavior can be computed rather than estimated from sampling.

A motivating discrepancy and its correct baseline. The repository’s initial $N = 2^{16}$ experiment used $m = 1024$ fixed-length chains of length $t = 64$, so $mt = N$. Treating the mt chain positions as independent samples would give the occupancy expression

$$1 - e^{-mt/N} = 1 - e^{-1} \approx 63.2\%.$$

That number is *not* the expected coverage of a Hellman matrix. Using the random-function recurrence of Ma and Hong [MH09], the same parameters predict approximately 16.651% coverage of the state space. The measured median exact coverage is approximately 16.8%. Thus the initial measurement is in close agreement with established random-function theory.

This distinction also exposes an important parameterization point. The standard Hellman matrix-stopping regime is expressed in terms of mt^2 relative to N , not merely mt/N . Our initial configuration satisfies $mt = N$ but $mt^2 = 64N$; it should therefore be treated as a finite-state stress point, not as a standard matrix-stopping configuration.

Contributions and scope. This paper is an experimental and reproducibility study, not a claim to have discovered chain coalescence or classical TMTO coverage theory. Its intended contributions are:

1. a precise, executable reduced-DES state model with canonical DES parity handling and an explicit reduction;
2. exact coverage and exact-recovery instrumentation for Hellman and bounded distinguished-point tables;
3. direct comparison of concrete cryptographic mappings with established random-function coverage predictions over controlled finite-state parameter sweeps;
4. independent Java and Rust reference implementations, with shared semantic fixtures and, before any timing comparison, a common committed experiment plan;
5. a reproducibility artifact in which numerical claims are generated from committed inputs, outputs, scripts, and environment metadata.

We make no claim to break full-size DES or AES, and no implementation in this repository is intended for production cryptography.

2 Related Work

Classical time–memory tradeoffs. Hellman introduced the cryptanalytic time–memory tradeoff for block-cipher key recovery and analyzed the effect of chain collisions under random-function

assumptions [Hel80]. Fiat and Naor later gave rigorous time/space tradeoffs for function inversion beyond the random-function setting [FN91]. Barkan, Biham, and Shamir developed a general model of cryptanalytic TMTOs based on stateful random graphs and proved rigorous bounds covering classical and rainbow-style constructions [BBS06]. Recent work continues to revisit the fundamental limits of preprocessing and function inversion [GGPSD23].

Random mappings and coverage. Iteration chains naturally induce functional graphs. Flajolet and Odlyzko systematically analyzed random-mapping statistics including cycles, components, and related structural parameters [FO90]. For Hellman tables specifically, Ma and Hong studied the expected coverage rate and supplied a recurrence that closely matches experiments under a random function model [MH09]. Their framework is a central baseline for the present paper: our goal is not to replace it, but to test it against exact coverage of a concrete finite cryptographic mapping.

Rainbow and perfect tables. Oechslin’s rainbow construction changes the reduction across chain positions, reducing the conditions under which collisions turn into permanent chain merges [Oec03]. Avoine, Junod, and Oechslin analyzed perfect classic, distinguished-point, and rainbow tables, including practical performance considerations [AJO08]. These works make clear that coverage, preprocessing cost, and lookup cost must be compared at matched success probabilities rather than through nominal table sizes alone.

False alarms and distinguished points. Endpoint matches do not always imply successful recovery: false alarms can force regeneration of candidate chains and materially affect the online cost. Hong analyzed this cost for Hellman and rainbow tradeoffs [Hon08]. Lee and Hong compared major perfect-table variants and developed detailed analysis for distinguished-point constructions [LH12].

Position of this work. The present paper focuses on reproducibility at finite state sizes where the entire mapping and table coverage can be enumerated. This permits a direct three-way comparison among established random-function predictions, a concrete DES-derived mapping, and independent Java/Rust implementations. The value of the study is therefore in exact instrumentation, controlled comparisons, and reproducible validation—not in reintroducing known TMTO phenomena.

3 Model and Definitions

This section fixes the finite-state model used by both implementations and by all exact-coverage measurements. The definitions deliberately separate the cryptographic primitive from the reduction and from the TMTO table structure.

3.1 Reduced effective DES state space

For an integer

$$1 \leq b \leq 28,$$

let

$$X_b = \{0, 1, \dots, 2^b - 1\}, \quad N = |X_b| = 2^b.$$

The parameter b counts *effective DES key bits*; it is not a count of raw bits in the eight-byte DES key representation.

DES stores eight bytes, with one parity bit in each byte. The experiment keeps the first four bytes fixed and allows the final four bytes to carry at most $4 \cdot 7 = 28$ effective data bits. Let

$$K_b : X_b \longrightarrow \{0, 1\}^{64}$$

denote the canonical encoding used by the artifact. The state is decomposed into four seven-bit groups, placed into the four variable DES bytes from most significant group to least significant group, and the least-significant bit of each byte is then chosen to give odd DES parity. Unused high state bits are zero. This convention makes the state-to-key map deterministic and invertible on its image.

The fixed four-byte prefix uses seven effective one-bits in each byte. Its purpose is solely to preserve the effective-data structure of the historical prototype; it is not a security parameter.

3.2 Encryption oracle and reduction

Let P be the fixed ASCII plaintext

HPC reproducibility fixture.

For $x \in X_b$, define the deterministic encryption oracle

$$G_b(x) = \text{DES}_{K_b(x)}(P),$$

using DES in ECB mode with PKCS#5 padding. DES is obsolete and appears here only as a small deterministic fixture whose entire reduced state space can be enumerated.

The reduction

$$R_b : \{0, 1\}^* \longrightarrow X_b$$

uses the final four ciphertext bytes. From each such byte it discards the least-significant bit, retains the upper seven bits, concatenates the resulting four seven-bit words into a 28-bit integer, and finally truncates to the low b bits. Equivalently, if the final four ciphertext bytes are c_0, c_1, c_2, c_3 , interpreted as unsigned integers, define

$$u_i = \lfloor c_i / 2 \rfloor \in \{0, \dots, 127\},$$

$$U = (((u_0 \cdot 2^7 + u_1) \cdot 2^7 + u_2) \cdot 2^7 + u_3),$$

and

$$R_b(c) = U \bmod 2^b.$$

The TMTO iteration map is therefore

$$F_b = R_b \circ G_b : X_b \longrightarrow X_b. \tag{1}$$

For fixed b and P , this is one deterministic functional graph on N vertices.

3.3 Targets and exact recovery

A trial chooses a target state

$$x^* \in X_b$$

and publishes only

$$y^* = G_b(x^*).$$

An algorithm reports exact recovery only when it returns a state $\hat{x}$ such that

$$G_b(\hat{x}) = y^*.$$

The implementation therefore verifies candidate states against the full target ciphertext rather than treating an endpoint collision as success.

For the deliberately reduced state spaces studied here, exhaustive search is a reference method: test $x = 0, 1, \dots, N - 1$ until the target ciphertext is matched. It has exact state-space coverage one and serves as a correctness control rather than as a practical attack on DES.

3.4 Hellman matrix and table

Choose m distinct start states

$$S = (s_1, \dots, s_m) \subset X_b.$$

For a chain length $t \geq 1$, define the Hellman matrix

$$H_{i,0} = s_i, \quad H_{i,j+1} = F_b(H_{i,j}), \quad 0 \leq j < t.$$

Thus row i contains $t + 1$ states

$$H_{i,0}, H_{i,1}, \dots, H_{i,t},$$

but the table represents the first t positions

$$H_{i,0}, \dots, H_{i,t-1}$$

and stores the pair

$$(H_{i,0}, H_{i,t}).$$

This convention matches the coverage definition used by Ma and Hong [MH09] and matches the artifact's exact coverage analyzer.

The implementation indexes all starts by their final endpoint. Multiple rows with the same endpoint are retained rather than silently overwritten.

3.5 Exact Hellman coverage

The represented-state set of one Hellman matrix is

$$\mathcal{C}_H = \{H_{i,j} : 1 \leq i \leq m, 0 \leq j < t\}. \quad (2)$$

Its exact covered-state count and state-space coverage fraction are

$$C_H = |\mathcal{C}_H|, \quad \rho_H = \frac{C_H}{N}.$$

The normalized coverage rate used in the classical literature is instead

$$\text{CR}_H = \frac{C_H}{mt}.$$

The two normalizations answer different questions:

$$\rho_H = \frac{mt}{N} \text{CR}_H.$$

When $mt = N$, as in the initial $b = 16, m = 1024, t = 64$ stress configuration, these two numerical values happen to coincide. They should not otherwise be conflated.

The artifact computes C_H exactly with a bit set over all N reduced states; this analysis is performed outside the timed preprocessing and lookup paths.

3.6 Endpoint collisions and candidate verification

Let

$$e_i = H_{i,t}$$

be the endpoint of row i . The number of distinct stored endpoints is

$$E_H = |\{e_1, \dots, e_m\}|.$$

Endpoint multiplicity is retained because several starts may produce the same endpoint.

During online lookup, the target ciphertext is reduced and projected forward to possible endpoint positions. If a projected value hits the endpoint index, all previously unchecked rows stored under that endpoint are regenerated from their starts. An endpoint hit that fails to reproduce y^* is a false candidate event. We therefore distinguish:

- *projected endpoint matches*: online projections that hit a stored endpoint key;
- *candidate rows checked*: distinct stored rows regenerated after such hits;
- *exact recovery*: regeneration finds a state whose full ciphertext equals y^* .

These counters measure concrete lookup work and should not be replaced by a success/failure bit alone.

3.7 Distinguished-point tables

Fix a difficulty parameter

$$1 \leq d \leq b$$

and define

$$D_d(x) = 1 \iff x \bmod 2^d = 0.$$

Thus a fraction exactly 2^{-d} of the state space is distinguished. Under a uniform-state heuristic, a fresh state is distinguished with probability

$$p_d = 2^{-d},$$

so an unconstrained geometric stopping model has mean

$$1/p_d = 2^d$$

transitions.

For a maximum chain length L , a distinguished-point row starts at s_i and iterates F_b until the first transition whose output is distinguished:

$$\tau_i = \min\{j \in \{1, \dots, L\} : D_d(F_b^j(s_i)) = 1\}.$$

If no such j exists, the row is *truncated* and is not stored. A successful row stores its start and distinguished endpoint

$$(s_i, F_b^{\tau_i}(s_i)).$$

The exact DP coverage used by the artifact is

$$\mathcal{C}_{\text{DP}} = \bigcup_{\tau_i \text{ exists}} \{F_b^j(s_i) : 0 \leq j < \tau_i\}.$$

In particular, truncated rows are excluded from this represented-state set. This convention is made explicit because other implementations may count partially generated truncated rows differently.

3.8 Measured quantities

For each parameter configuration and trial, the study records as available:

state-space size	N ,
exact coverage	C/N ,
exact recovery	$\mathbf{1}[\hat{x} \text{ recovered}]$,
stored rows	number of accepted table rows,
distinct endpoints	E ,
truncated DP rows	T_{trunc} ,
endpoint/projection hits	Q_{end} ,
candidate rows regenerated	Q_{cand} ,
offline time	T_{off} ,
online time	T_{on} .

Later parameter sweeps additionally record functional-graph statistics such as row merges, self-intersections, and cycle/component information. Those quantities are reported only after the corresponding instrumentation is committed.

4 Random-Function Coverage Baselines

The concrete map F_b in Eq. (1) is deterministic. Classical TMTO analyses nevertheless model the underlying iteration function as a random mapping on a set of size N . This section states the random-function baseline against which our exact DES-derived measurements are compared.

4.1 Why independent occupancy is not a Hellman prediction

If q points were sampled independently and uniformly from an N -element set, the expected number of distinct observed states would be

$$N \left(1 - \left(1 - \frac{1}{N} \right)^q \right) \approx N(1 - e^{-q/N}).$$

Putting $q = mt$ gives the familiar occupancy expression

$$\rho_{\text{iid}} \approx 1 - e^{-mt/N}. \quad (3)$$

For $mt = N$, this is $1 - e^{-1} \approx 0.6321$.

Equation (3) is useful as a deliberately naive reference, but Hellman matrix positions are not independent samples. Within each row, later entries are deterministic iterates of earlier entries; once two trajectories meet under a fixed function, their suffixes coincide. Classical Hellman analysis and later random-mapping work explicitly account for this dependence [Hel80, FO90, MH09]. We therefore do not use Eq. (3) as the expected Hellman coverage.

4.2 Ma–Hong expected coverage recurrence

Ma and Hong [MH09] analyze the first t columns of an $m \times (t + 1)$ Hellman matrix. Let

$$\overline{H}_k = \{H_{i,j} : 1 \leq i \leq m, 0 \leq j \leq k\}$$

and let p_k denote, after normalization by N , the expected number of states appearing for the first time in column k . Introducing

$$s_k = \sum_{j=0}^{k-1} p_j,$$

their large- N approximation yields

$$s_0 = 0, \quad s_{k+1} = 1 - \exp\left(-\frac{m}{N}\right) \exp(-s_k). \quad (4)$$

The expected coverage rate relative to the mt nominal matrix positions is

$$\text{ECR}(N, m, t) = \frac{N}{mt} s_t. \quad (5)$$

Consequently, the predicted fraction of the *whole state space* covered by the first t columns is simply

$$\rho_{\text{MH}}(N, m, t) = \frac{mt}{N} \text{ECR}(N, m, t) = s_t. \quad (6)$$

This distinction in normalization is particularly useful in our experiments. The artifact reports C_H/N , whereas much of the TMTO literature reports coverage relative to mt .

4.3 Initial configuration

For

$$N = 2^{16} = 65536, \quad m = 1024, \quad t = 64,$$

Eq. (4) gives

$$s_{64} \approx 0.1665143030.$$

Thus random-function theory predicts approximately

$$0.1665143030 N \approx 10912.68$$

distinct represented states, or 16.6514% of the state space.

The committed Java and Rust experiments report median exact Hellman coverage near 16.8%. The purpose of the larger sweep is to determine whether this agreement persists across state dimensions and chain shapes, not merely at one configuration.

For contrast, Eq. (3) gives

$$1 - e^{-1} \approx 63.2121\%$$

because $mt = N$. The difference between 63.2% and 16.65% is therefore not itself a novel empirical anomaly; it is what established fixed-map Hellman coverage theory already leads us to expect.

4.4 The matrix-stopping parameter

Hellman's standard parameterization is often discussed under the *matrix-stopping rule*

$$mt^2 \approx N, \quad (7)$$

rather than $mt \approx N$ [Hel80, MH09]. Under the exact condition $mt^2 = N$, Eq. (4) becomes

$$s_{k+1} = 1 - \exp(-1/t^2) \exp(-s_k),$$

and

$$\text{ECR}(t) = t s_t.$$

Our initial configuration instead satisfies

$$mt = N, \quad mt^2 = 64N.$$

It is therefore a useful high-overlap finite-state stress point, but it should not be described as a standard matrix-stopping configuration. The experiment matrix includes both regimes:

1. configurations near $mt^2 = N$, to connect directly with classical parameterization;
2. equal- mt chain-shape sweeps, to isolate the effect of changing the number and length of rows at fixed nominal transition budget;
3. the historical/repository stress configuration $m = 1024, t = 64$ at $N = 2^{16}$.

4.5 Finite- N and model limitations

Equation (4) is an approximation derived under a random-function model. It is not an exact identity for an arbitrary fixed mapping. Two sources of deviation are therefore relevant to the present study.

First, the reduced state spaces are finite enough that the entire functional graph can be enumerated. Small- N corrections, self-intersections, and specific component structure need not equal their asymptotic expectations. Ma and Hong themselves discuss a special discrepancy for the $m = 1$ case and give a separate exact expression [MH09].

Second, F_b is not sampled uniformly from the set of all functions $X_b \rightarrow X_b$: it is built from DES encryption, a fixed plaintext, a structured key embedding, and a deterministic reduction. Agreement with Eq. (6) is therefore an empirical property to be tested, not an assumption.

These observations motivate a direct random-map control. For every state dimension and TMTD parameter set used in the final experiments, we will compare the DES-derived map with independently generated random mappings

$$F_{\text{rand}} : X_b \longrightarrow X_b$$

under the same start states and table parameters.

4.6 Distinguished-point baseline

For the predicate

$$D_d(x) = 1 \iff x \bmod 2^d = 0,$$

exactly $N/2^d$ states are distinguished. If successive states behaved as fresh uniform samples, the first distinguished hit would have geometric parameter

$$p = 2^{-d}.$$

For an unbounded chain,

$$\Pr[\tau = k] = (1 - p)^{k-1}p, \quad \mathbb{E}[\tau] = 2^d.$$

With a hard transition bound L , the fresh-sample truncation probability is

$$\Pr[\tau > L] = (1 - 2^{-d})^L. \tag{8}$$

This is again a baseline rather than an exact statement for iterations of a fixed finite mapping. Cycles that contain no distinguished state can make a row fail to terminate even when L is large, and merged trajectories create dependence among rows. The final experiments compare measured truncation and coverage with Eq. (8) and with exact functional-graph structure.

4.7 What constitutes a meaningful deviation

For each configuration we compute the theory residual

$$\Delta_{\text{cov}} = \rho_{\text{measured}} - \rho_{\text{MH}}.$$

A single nonzero residual is not evidence of cryptographic structure: finite-size variability and the approximation itself must be accounted for. The final paper therefore reports distributions over repeated random start sets and, for the random-map controls, over independently sampled mappings. Only a deviation that is stable relative to these controls will be interpreted as evidence that the DES-derived map/reduction has coverage behavior not captured by the random-function model.

5 Reference Implementations

The artifact contains two independently written implementations of the finite state model in Section 3. Their purpose is not merely to provide two programming-language front ends. Agreement between independent implementations is used as evidence that the observed table statistics follow from the specified TMTO semantics rather than accidental behavior of one code base.

5.1 Java 17 reference

The Java implementation uses the standard Java cryptography interface for `DES/ECB/PKCS5Padding`. The reduced-state layer constructs the eight-byte DES key explicitly, inserts odd parity in every byte, and varies only the effective data bits defined in Section 3.

The core components are `ReducedDesKeySpace`, `DesOracle`, `LegacyReduction`, `HellmanTable`, `DistinguishedPointTable`, and `CoverageAnalyzer`. Endpoint collisions are retained, and online lookup always verifies the full target ciphertext.

5.2 Rust reference

The Rust crate implements the same semantics independently. It uses the RustCrypto `des` crate for the DES block primitive and explicitly performs PKCS#5-compatible padding before ECB-style block encryption. State encoding, parity, reduction, Hellman construction, distinguished-point construction, candidate regeneration, and exact coverage are separately implemented in Rust.

The independence matters because equality requires agreement on byte order, padding, parity, reduction bit positions, coverage conventions, truncated DP rows, endpoint collisions, and candidate verification.

5.3 Literal shared experiment plan

Small committed vectors already compare state encoding, key bytes, ciphertexts, and reductions. For table-level equivalence we use the stronger workload

`experiments/plans/paper-b16-shared/`.

It contains literal CSV files specifying every target, every Hellman start, every distinguished-point start, and all common parameters.

The canonical workload uses

$$b = 16, \quad N = 65536, \quad m_H = 1024, \quad t_H = 64,$$

and

$$m_D = 1024, \quad d = 6, \quad L = 256$$

over 30 trials.

The literal plan removes a confounder present in the first benchmark runs. The Java harness generated trials with `SplittableRandom`, while the first Rust harness used `SplitMix64`. Both runs were deterministic, but they were not identical workloads. The committed CSV files now define the workload; neither language’s PRNG participates in the equivalence experiment.

5.4 Algorithmic equivalence output

For every trial both runners emit deterministic fields: target state and ciphertext, exact Hellman coverage, distinct endpoints, recovery state, endpoint matches and regenerated candidates, as well as exact DP coverage, stored/truncated rows, endpoint counts, recovery state, endpoint candidates, candidate rows, and projection steps.

No timing field is included. The Java and Rust semantic CSV files are required to be byte-for-byte identical before a cross-language timing experiment is admitted into the paper.

5.5 Separation of correctness and timing

The experimental order is

$$\text{semantic equivalence} \longrightarrow \text{identical workload} \longrightarrow \text{performance comparison}.$$

Exact coverage is also computed outside timed preprocessing and lookup paths. The current section treats deterministic Java/Rust equality as correctness evidence; performance is considered only after that gate.

6 Experimental Methodology

The experiments are designed to separate three questions that are easy to conflate: whether the implementations agree, whether the concrete DES-derived mapping follows random-function TMTO theory, and how wall-clock performance depends on implementation language. The first question is resolved before the second is measured, and both are resolved before the third is interpreted.

6.1 Semantic-equivalence gate

Section 5 describes the committed 30-trial literal workload consumed by Java and Rust. The resulting deterministic CSV files are required to be byte-for-byte identical. Timing fields are deliberately absent from this gate. Thus any later performance comparison starts from identical targets, starts, table parameters, ciphertexts, coverage, endpoint counts, and recovery decisions.

6.2 Exact-map experiment rather than a timing benchmark

The coverage experiment is not timed cryptographic benchmarking. For each

$$b \in \{12, 14, 16, 18\},$$

the Rust experiment harness first enumerates the entire concrete mapping

$$F_b : X_b \rightarrow X_b$$

once and stores its next-state array. Every later table traversal for that state dimension is then performed over this exact array. This has two advantages: coverage experiments do not repeatedly pay DES implementation cost, and the resulting measurements depend only on the finite functional graph and the selected starts.

The complete mapping is feasible to enumerate at these dimensions: the largest space used in this paper has only

$$2^{18} = 262144$$

vertices.

6.3 Random-mapping control

For every state dimension and trial we also generate one independent pseudo-random control mapping

$$F_{\text{rand}} : X_b \rightarrow X_b$$

by assigning every vertex an independent SplitMix64-generated output reduced modulo N . SplitMix64 is used only as a deterministic experiment generator; no cryptographic pseudorandomness claim is required.

Within a fixed trial and parameter configuration, the DES-derived mapping and the random control receive exactly the same sorted set of start states. Consequently,

$$\Delta_{\text{map}} = \rho_{\text{DES}} - \rho_{\text{rand}}$$

does not contain start-set variation for that trial.

The random mapping changes across trials. The DES-derived F_b is fixed for a given b , while its selected start set changes. This distinction is intentional: the random controls estimate the variability across random functions, whereas the DES measurements characterize repeated table constructions on one concrete cryptographic functional graph.

6.4 Hellman parameter families

We use two complementary parameter families. The first consists of matrix-stopping configurations satisfying

$$mt^2 = N.$$

The second holds the nominal represented-position budget fixed at

$$mt = N$$

for $b = 16$ while varying the chain shape. The original repository stress configuration

$$(m, t) = (1024, 64)$$

is included in this second family.

The theory-only configurations and their Ma-Hong predictions are generated from committed code before measurements are run. The resulting grid is:

Configuration	b	m	t	mt/N	ρ_{MH}
stop-b12-a	12	64	8	0.1250	10.444%
stop-b12-b	12	16	16	0.0625	5.303%
stop-b14-a	14	64	16	0.0625	5.303%
stop-b14-b	14	16	32	0.0312	2.671%
stop-b16-a	16	256	16	0.0625	5.303%
stop-b16-b	16	64	32	0.0312	2.671%
stop-b16-c	16	16	64	0.0156	1.341%
stop-b18-a	18	256	32	0.0312	2.671%
stop-b18-b	18	64	64	0.0156	1.341%
stop-b18-c	18	16	128	0.0078	0.671%
budget-b16-t16	16	4096	16	1.0000	31.169%
budget-b16-t32	16	2048	32	1.0000	22.951%
stress-b16-current	16	1024	64	1.0000	16.651%
budget-b16-t128	16	512	128	1.0000	11.985%

Table 1: Hellman parameter configurations. The last column is the Ma–Hong random-function prediction for the fraction of the whole state space covered by the first t columns.

6.5 Repeated trials and deterministic seeds

Each configuration is evaluated for 30 trials on the DES-derived mapping and 30 matched random-function controls. Start sets are distinct within a table and are generated deterministically from a documented base seed, state dimension, trial index, and configuration index. The seed derivation is part of the committed experiment source. Re-running the same commit therefore reconstructs the same workloads and random maps.

The shared Java/Rust equivalence plan is independent of this sweep generator. Its literal CSV files remain the canonical evidence for cross-language semantic equality.

6.6 Primary Hellman observables

For each table we record:

C_H = number of distinct represented states,

$\rho_H = C_H/N$,

E_H = number of distinct endpoints,

$D_H = mt - C_H$.

The quantity D_H is a compact count of repeated nominal matrix positions; it does not distinguish self-intersections from merges between different rows.

For every configuration we also record the theory residual

$$\Delta_{\text{MH}} = \rho_H - \rho_{\text{MH}}(N, m, t),$$

where ρ_{MH} is computed from Eq. (6). The independent occupancy quantity

$$\rho_{\text{iid}} = 1 - e^{-mt/N}$$

is retained only as a reference column, not as the Hellman null model.

6.7 Interpretation rule

A close match between one DES-derived table and the Ma-Hong recurrence is not by itself evidence that the concrete mapping behaves randomly. The paper therefore compares distributions across start sets with distributions across independently generated random maps. We interpret a DES-vs-theory residual as structurally interesting only if it is stable relative to the random-map control distribution.

Similarly, changing t at fixed mt simultaneously changes row count, chain length, endpoint structure, and the amount of permanent coalescence. Results from that family are interpreted as chain-shape effects rather than as differences in nominal precomputation budget.

6.8 Distinguished-point experiments

The committed shared plan already measures exact DP coverage, stored/truncated rows, endpoint multiplicities, recovery, and projection work for

$$d = 6, \quad L = 256.$$

The final DP sweep varies d and L and compares the measured truncation rate with Eq. (8). Those measurements are kept in a separate output table so that the Hellman random-map comparison can be audited independently.

7 Results

8 Discussion and Limitations

9 Reproducibility

10 Conclusion

References

- [AJO08] Gildas Avoine, Pascal Junod, and Philippe Oechslin. Characterization and improvement of time-memory trade-off based on perfect tables. *ACM Transactions on Information and System Security*, 11(4), 2008.
- [BBS06] Elad Barkan, Eli Biham, and Adi Shamir. Rigorous bounds on cryptanalytic time/memory tradeoffs. In *Advances in Cryptology – CRYPTO 2006*, volume 4117 of *Lecture Notes in Computer Science*, pages 1–21. Springer, 2006.
- [FN91] Amos Fiat and Moni Naor. Rigorous time/space tradeoffs for inverting functions. In *Proceedings of the 23rd Annual ACM Symposium on Theory of Computing*, pages 534–541. ACM, 1991.
- [FO90] Philippe Flajolet and Andrew M. Odlyzko. Random mapping statistics. In *Advances in Cryptology – EUROCRYPT ’89*, volume 434 of *Lecture Notes in Computer Science*, pages 329–354. Springer, 1990.

- [GGPSD23] Alexander Golovnev, Siyao Guo, Spencer Peters, and Noah Stephens-Davidowitz. Revisiting time-space tradeoffs for function inversion. In *Advances in Cryptology – CRYPTO 2023, Part II*, volume 14082 of *Lecture Notes in Computer Science*, pages 453–481. Springer, 2023.
- [Hel80] Martin E. Hellman. A cryptanalytic time–memory trade-off. *IEEE Transactions on Information Theory*, 26(4):401–406, 1980.
- [Hon08] Jin Hong. The cost of false alarms in hellman and rainbow tradeoffs. Cryptology ePrint Archive, Paper 2008/362, 2008.
- [LH12] Ga Won Lee and Jin Hong. A comparison of perfect table cryptanalytic tradeoff algorithms. Cryptology ePrint Archive, Paper 2012/540, 2012.
- [MH09] Daegun Ma and Jin Hong. Success probability of the hellman trade-off. *Information Processing Letters*, 109(7):347–351, 2009.
- [Oec03] Philippe Oechslin. Making a faster cryptanalytic time-memory trade-off. In *Advances in Cryptology – CRYPTO 2003*, pages 617–630. Springer, 2003.